macro

Seven Seas A-10

Security Audit

July 10, 2024
Version 1.0.0

Presented by 0xMacro

Table of Contents

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled ‘Source Code’. The security audit was performed by the Macro security team on periodically from june 20th to july 8th 2024.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro’s review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Low	2	-	-	2

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Seven Seas team.
- Available documentation in the repository.

Trust Assumptions:

Part of this audit investigated authorizing trusted protocols to call bulk deposit on the teller. Doing so allows the protocol to skip over the set share lock period that normal depositors experience. It is trusted that the protocols authorized to make bulk deposits cannot or are trusted to not maliciously use the lack of a share lock period, to arbitrage share price updates.

Source Code

The following source code was reviewed during the audit:

- **Repository:** boring-vault
- **Commit Hash:** `35433417a394c49538260f30b5d37ab3e8de99d1`

Specifically, we audited the following contracts within this repository.

Source Code	SHA256
src/base/DecodersAndSanitizers/BridgingDecoderAndSanitizer.sol	`bd6d9dfa44f3eb1892b0fd1865556918d77b5660d18fc729c37b79e859f2a531`
src/base/DecodersAndSanitizers/EtherFiLiquidEthDecoderAndSanitizer.sol	`1d4d53ddb56cbba71cd64f581910237977bb87968eaba625fa8a1ac19e8816c0`
src/base/DecodersAndSanitizers/PancakeSwapV3FullDecoderAndSanitizer.sol	`2f44f23d7c1a01701e4446d0e7eb83510d079a4ed3dce055b83108caea3dd053`
src/base/DecodersAndSanitizers/Protocols/CCIPDecoderAndSanitizer.sol	`df112acbbe4c3bc48645abdc008eaf0468e45575cab336453939a6079208f4ca`
src/base/DecodersAndSanitizers/Protocols/ITB/ITBPositionDecoderAndSanitizer.sol	`338232d68727bbc8c8f790995a97de8b6bc139a08b57e32614f8562d42a8e3c2`
src/base/DecodersAndSanitizers/Protocols/ITB/karak/KarakDecoderAndSanitizer.sol	`ded26ce6189c2acdbbe93c7dd0b8e59027ee0e5e5ebbb3c332bc6e9f4b27cf88`
src/base/DecodersAndSanitizers/Protocols/PancakeSwapV3DecoderAndSanitizer.sol	`c8bd2c24f98cbfef98ca7fa15396054e9ffb4217bc9ca5f09c6f6b2a688183a1`
src/base/DecodersAndSanitizers/Protocols/PendleRouterDecoderAndSanitizer.sol	`c2e1bec097d8ba7038371eb239125f28277ca310890dad8b1cfe6768c0f26606`

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/ArbitrumNativeBridgeDecoderAndSanitizer.sol	`29c261d40412922b7a9288c89533a08cfde065c3cb153ccdb68cc6efdd7643fc`
src/base/Roles/CrossChain/CrossChainTellerWithGenericBridge.sol	`cd398597495709603bd365194aacf8351433a00d6ece76ea15e89099077a860f`
src/base/Roles/TellerWithMultiAssetSupport.sol	`07884e603f6c8ac9148cd3bcdda9d9bf1cd94b089d8b272b452494c7143e8894`
src/interfaces/DecoderCustomTypes.sol	`731408dc5444a13052f61d2fd5501feb2dce900b30f9fd27deb86575d88cc6f5`

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:
    - How bad things can get (for a vulnerability)
    - The significance of an improvement (for a code quality issue)
    - The amount of gas saved (for a gas optimization)
2. The high/medium/low **likelihood** of the issue:
    - How likely is the issue to occur (for a vulnerability)
3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client **must** fix the issue, no matter what, because not fixing would mean **significant funds/assets WILL be lost.**
(H-x) High	We recommend the client **must** address the issue, no matter what, because not fixing would be very bad, *or* some funds/assets will be lost, *or* the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to **seriously consider** fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albiet not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

~~L-1~~ Bulk deposits and withdrawals are not pauseable

TOPIC	STATUS	IMPACT	LIKELIHOOD
protocol design	Fixed	Low	Low

In **TellerWithMultiAssetSupport.sol, the `deposit()` and `depositWithPermit()` functions are prevented if the teller is paused, however the `bulkDeposit()` and `bulkWithdraw()` functions are not restricted when the teller is paused. The initial intent was that these bulk functions would only be called by trusted entities, like solvers, so it made sense to allow for these functions to execute even when paused. In the case of `bulkDeposit()`, it skips over setting a share lock for the address receiving the shares, which is a desired outcome for some trusted protocols that want to integrate with these vaults. If authorization to call `bulkDeposit()` is given to these protocols, it makes sense for these functions to be paused as well, since interaction is no longer solely operated by limited entities, and pausing these functions may be required.**

Remediations to Consider

Add checks if the teller is paused for both `bulkDeposit()` and `bulkWithdraw()`.

~~L-2~~ CCIP events benefit from being indexed

TOPIC	STATUS	IMPACT	LIKELIHOOD

Events	Fixed	Low	Medium

In `CrossChainTellerWithGenericBridge.sol` the events MessageSent and MessageReceived are emitted when sending or receiving a message:

```
event MessageSent(bytes32 messageId, uint256 shareAmount, address to);
event MessageReceived(bytes32 messageId, uint256 shareAmount, address to);
```

Reference: **CrossChainTellerWithGenericBridge.sol#L13-L14**

The `messageId` and `to` address are not `indexed`, which may make it more difficult for off chain application to get relevant bridging information for a user or messageId.

Remediations to Consider

Add `indexed` to the `messageId` and `to` event params.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.